7. Project Documentation Phase

FitBuddy – AI Fitness Plan Generator using Gemini Models

Phase Report

Prepared from the supplied FitBuddy project documentation and organized into a project lifecycle deliverable.

1. Project Summary

FitBuddy is a web-based AI fitness plan generator using FastAPI, Google Gemini models, Jinja2 templates, SQLAlchemy, and SQLite.

2. Features

Personalized seven-day workout plans; nutrition/recovery tips; feedback-based plan updates; persistent user and plan storage; admin view of registered users and original/updated plans; interactive API documentation.

3. Technology Stack

Python; FastAPI; Uvicorn; HTML/CSS; Jinja2; SQLAlchemy; SQLite; Google Gemini API; python-dotenv for environment configuration.

4. Installation

Open a terminal in the project root. Create a virtual environment using python -m venv venv. Activate it on Windows using venv\Scripts\Activate.ps1. Install dependencies using pip install -r requirements.txt. Create .env from .env.example and add a valid Gemini API key.

5. Run Instructions

Run uvicorn app.main:app --reload. Open http://127.0.0.1:8000 in a browser. Use http://127.0.0.1:8000/docs to inspect and test the API endpoints.

6. Main Routes

GET / — home form. POST /generate-workout — generate a workout plan and tip. POST /submit-feedback — revise an existing plan. GET /view-all-users — admin list. GET /docs — API documentation.

7. Configuration and Troubleshooting

If requirements.txt is missing, ensure the terminal is in the project root and create the file before installing. If activation is blocked in PowerShell, use Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass for the current terminal session. If AI generation fails, check the API key, network connection, model availability, and quota.

8. Limitations and Responsible Use

AI-generated fitness content can be inaccurate or unsuitable for some individuals. Users should adapt activities to their abilities and seek qualified advice for injuries, medical conditions, or specific dietary needs. Avoid storing unnecessary personal information.

9. Maintenance

Keep dependencies updated, rotate exposed API keys, back up the SQLite database if needed, and rerun tests after code or model changes.

Note: This document describes the intended design and workflow. Test outcomes should be recorded only after the application has been run and verified.